

Epigenetic Language Models: Lifelong Learning by Intercalating Knowledge into Structural Zeros

A mechanism study of Free-Slot Adaptation

Parallax / Chutes.ai

October 9, 2026

AI-assisted draft. Generated with AI assistance and revised through repeated adversarial review; the experiments were run and checked by the authors. It is a *mechanism study*: all results are a **dense fp32** synthetic-assay demonstration of a freezing mechanism. **pair8**/ternary never enters the adaptation loop, and no real model is evaluated in this paper. The personal-model use case (§6) is an *untested proposal*, not a result. Code: github.com/chutesai/epigenesis.

Abstract

We call a model *epigenetic* if what it learns after deployment never edits its base weights: acquired knowledge lives in a separate sparse layer that can be removed, restoring the original model exactly. A **pair8**-quantized network holds at least half of its weights at structural zero. We study whether those zeros can serve as a continual-learning *reserve*: freeze the learned weights, write new knowledge only into the zero slots, and recover the old behavior with a per-slice mask. This is exactly Mallya and Lazebnik [2018], placed in a format whose zeros are already addressable. On a synthetic knowledge-capacity assay we quantify the *mechanism* (dense fp32, random mask): the masked base is preserved exactly (by construction), the reserve memorizes random facts up to a measured maximum and then declines, a mask-free “soft” variant degrades rather than stays flat, and—counted in the representation we actually train—tuned LoRA is *both more capable and more storage-efficient*. The contribution is therefore a careful, honest characterization of the mechanism and its accounting, plus a scoped proposal: a *revocable domain patch* inside a quantized model, whose natural application—a private, continually adapting model—we motivate but do not claim to have built. We state precisely what is untested and the single experiment (ternary reserve through the real quantizer) that would resolve whether a storage advantage exists—an outcome we argue is genuinely open, not forecastable from the format assay.

1 Introduction

Adding knowledge to a trained model usually means adding parameters—a LoRA adapter [Hu et al., 2022], an extra expert, or a full fine-tune that risks forgetting. We ask a narrower question: an extreme-quantized model has already paid for a pool of weight slots held at zero; can those zeros double as a place to put new knowledge?

Free-Slot Adaptation (FSA). Store a model in format Φ (here **pair8**: ternary weights, at most two of every four aligned pairs non-zero). Let $\mathcal{S}_0$ be the non-zero support and $\mathcal{Z}_0$ the zero reserve. To learn a new domain d : freeze $\mathcal{S}_0$ and shared parameters, train a disjoint slice $\mathcal{R}_d \subseteq \mathcal{Z}_0$, and store a mask M_d . At inference the base uses $\mathcal{S}_0$; domain d uses $\mathcal{S}_0 \cup \mathcal{R}_d$. The mechanism is Mallya and Lazebnik [2018]’s; the only new element is that the free weights come from the compression format rather than a pruning pass.

What we claim, and what we do not. We claim a clean characterization of this mechanism on a synthetic assay: it preserves the masked base exactly, it memorizes a bounded amount of new material, and its cost is a *trade against compression*, not free capacity. We do *not* claim it

beats parameter-efficient adaptation—our own tuned-LoRA results show the opposite—nor that it is validated on a real model or in the ternary format it is motivated by. We are explicit about this throughout, because an earlier draft overstated it and adversarial review corrected us.

Contributions. (1) The framing of quantization zeros as a continual-learning reserve, with honest accounting of what is free (logical parameter count, dense-kernel FLOPs) and what is not (storage, which trades against compression; the mask; the scales). (2) A synthetic assay isolating the mechanism: exact masked retention, a measured reserve ceiling and interference regime, a mask-free soft variant, and a indicative (uncontrolled) LoRA comparison that we report even though it is unfavorable. (3) A scoped proposal—a revocable domain patch inside a quantized model—and an honest discussion of where it could matter (a private, continually-learning model) and where retrieval already wins. (4) The gating experiment (ternary reserve through the real quantizer) whose outcome is open.

2 Background and related work

Mask-based continual learning. PackNet [Mallya and Lazebnik, 2018] freezes kept weights and trains freed ones, recovering a task by its mask; Piggyback [Mallya et al., 2018] *learns masks over fixed weights* (it does not refill zeros); HAT [Serra et al., 2018] uses hard task-attention and reports *reduced*, not zero, forgetting; Supermasks [Zhou et al., 2019] make the mask itself the trainable object; Progressive Networks [Rusu et al., 2016] add and could drop columns. FSA is PackNet with the free support supplied by the quantization format. **Quantized continual learning.** Ada-QPackNet [Pietroni et al., 2023] and TinySubNets [Wójcik et al., 2025] already combine PackNet-style capacity reuse with quantization; SPDF [Thangarasa et al., 2023] reopens zeroed pretrained weights for dense fine-tuning. These sharply narrow our novelty; our distinct element is a reserve *pre-addressed by the deployed format* and composed by union. **Parameter-efficient and continual adapters.** LoRA [Hu et al., 2022], QLoRA [Dettmers et al., 2023], QA-LoRA [Xu et al., 2024]; Biderman et al. [2024] find LoRA learns and forgets less than full fine-tuning (a possible rank explanation, not a theorem); O-LoRA [Wang et al., 2023] does continual learning in orthogonal LoRA subspaces. **Sparse / composable adaptation.** Diff pruning [Guo et al., 2021] and composable sparse fine-tuning [Ansell et al., 2022]; union of *disjoint* sparse supports is the easy case of the latter. **Editing, memory, unlearning.** MEMIT [Meng et al., 2023] edits facts in place and can override; FSA adds paths that can suppress or correct outputs while enabled but cannot edit a base weight; GRACE [Hartvigsen et al., 2023] and WISE [Wang et al., 2024] keep deletable, routed side-memory for sequential edits while freezing the base; SISA [Bourtole et al., 2021] is exact unlearning by shard isolation; TOFU [Maini et al., 2024] benchmarks fictitious-fact forgetting. **Personalization and retrieval.** OPPU [Tan et al., 2024] attaches a per-user PEFT module (removed by user identity, not per record); LaMP [Salemi et al., 2024] is a personalization benchmark with retrieval baselines; RAG [Lewis et al., 2020] and kNN-LM [Khandelwal et al., 2020] store facts externally and delete them by dropping a record—the baseline any “delete a fact” claim must beat. **Low-bit models.** BitNet b1.58 [Ma et al., 2024, 2025] and LittleBit [Lee et al., 2025]; our assay methodology follows knowledge-capacity scaling [Allen-Zhu and Li, 2024].

3 Method

3.1 Format and reserve

In `pair8`, weights form aligned blocks of eight (four pairs); each weight is ternary times a scale, and at most two pairs per block are non-zero. A block thus has 417 reachable states ($1 + 4 \cdot 8 + \binom{4}{2} 8^2$), i.e. $\log_2 417/8 \approx 1.088$ bits/weight asymptotically (1.125 at a fixed 9 bits/block), and is $\geq 50\%$ zero

by construction (ternary alone guarantees no such fraction). **Caveat on the reserve size.** At a density that already uses both allowed pairs in a block, that block has *no in-codebook room* for a third pair. So the writable reserve is not the full $(1 - \rho)N$; either the base must leave pairs unused, or a patch must leave the `pair8` codebook and live in a *sidecar* of (`block`, `pair`, `trit`, `scale`) entries that the kernel applies or skips. A pre-assigned ternary reserve pair costs $\approx \log_2 3 = 1.58$ bits/weight, not 1.088 (that figure is the joint entropy spread over the forced zeros, not the marginal price of a filled slot).

3.2 Adaptation and the soft variant

Hard FSA, domain d : freeze $W[\mathcal{S}_0]$, all shared parameters, and—crucially for the real format—the discrete pair occupancy and the per-row scales; train only $W[\mathcal{R}_d]$ ($\mathcal{R}_d \cap \mathcal{S}_0 = \emptyset$) with the base live; store M_d . Inference: base $\rightarrow \mathcal{S}_0$; domain $\rightarrow \mathcal{S}_0 \cup \mathcal{R}_d$ (needs a task ID). **Soft FSA** removes the task ID: freeze the base, train the reserve with a KL penalty keeping base-input outputs unchanged, and run unmasked. This is Learning-without-Forgetting [Li and Hoiem, 2017] on a frozen disjoint support, with full base-input replay—a substantial rehearsal resource we note explicitly.

3.3 Honest accounting

FSA adds **zero new logical parameters** (same element count) and **zero extra FLOPs on a dense GEMM**—but the latter is kernel-conditional: a pair-sparse kernel (the kind the format exists to enable) does *more* work once zeros are filled. It is **not free in storage**: the deployable form is a frozen base pack plus a sidecar (new stored state) whose cost is the trits written (≈ 1.58 bits/weight), their addresses, and any scale overrides, plus the mask (1 bit/weight/domain, or ≈ 0 only with a fixed structural/hash rule—a 1-bit mask over 100B weights is 12.5 GB). At densities that already use the two allowed pairs in a block, the codebook is full, so a filled patch is *added bytes* on a packed base (a sidecar), not a reallocation of the base’s own compression budget. Freezing the shared scales is also a *capacity cut*: a reserve trit can only take values $\{-\alpha, 0, +\alpha\}$ for the base row’s per-row absmean scale α .

4 Theory

Base invariance (by construction, narrow). Restrict the forward to M_0 . If adaptation changes only $W[\mathcal{R}_d]$ with $\mathcal{R}_d \cap \mathcal{S}_0 = \emptyset$ and freezes all shared state, the masked base output is unchanged—an invariance of freezing, which is why measured “retention = 1.000” is a unit test that the mask was applied (it would hold even if the reserve learned nothing). *Three subtleties:* (i) it is the *masked* base; with the reserve enabled, the base behavior is not preserved (we measure strongly negative unmasked retention). (ii) In the real `pair8` format the decoded weight is $\alpha \cdot \text{trit}$ and the quantizer recomputes the pair occupancy and scale each forward; invariance then requires freezing *three* objects—the masters, the occupancy, and the scales—not just the masters. (iii) The soft variant is outside this argument entirely.

Reserve capacity is accounting, not a law. We do not vary ρ or the per-slot cost, so the curve is a memorization curve, not a test of a capacity expression. We therefore report $K_{\text{reserve}} \approx \kappa_{\text{adapt}} B_{\text{patch}}$ with B_{patch} the measured patch *bits* and κ_{adapt} an *observed* knowledge-bits-per-stored-bit efficiency conditional on base, support, scales, and training budget—and note it has no term for the interference decline we plot.

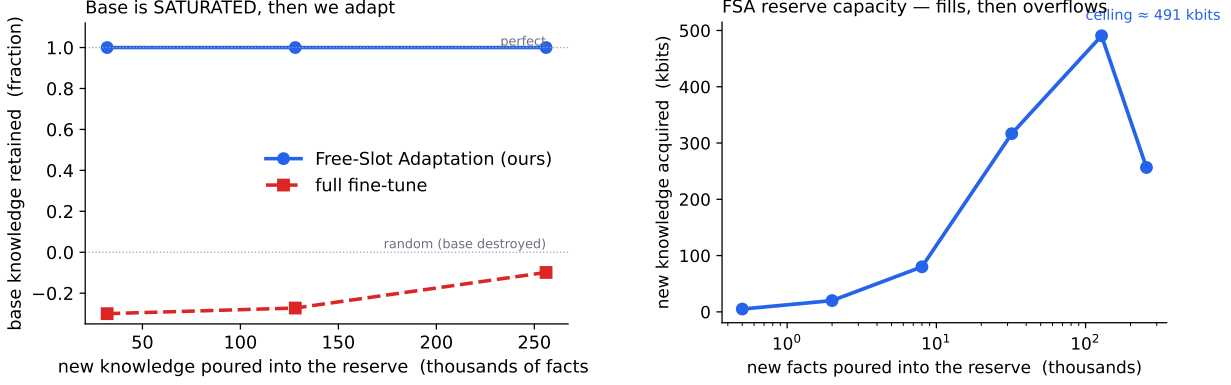
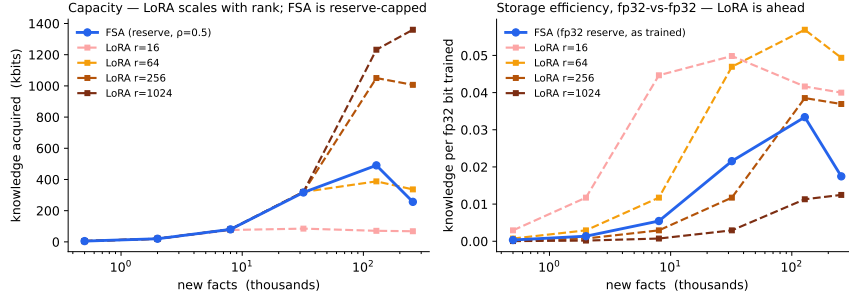


Figure 1: **Left:** masked base retention stays 1.000 (by construction) while full fine-tuning destroys the base. **Right:** the reserve memorizes up to ≈ 490.6 kbits, then *declines* to 256.5 kbits at 256k facts—a maximum under this training budget (fewer exposures per fact as the count grows can contribute), not a proven intrinsic ceiling.



Counted in the representation actually trained (fp32), LoRA is more efficient. A ternary-reserve win is a PREDICTION contingent on the untested Gate 0 — genuinely open (the format assay measures from-scratch QAT, not a frozen-base reserve).

Figure 2: **FSA vs. tuned LoRA.** Left: LoRA’s capacity scales with rank and *exceeds* FSA’s reserve maximum at $r \geq 256$. Right: counted in the representation actually trained (fp32), LoRA is *more* storage-efficient ($\kappa \approx 0.057$ at $r=64$ vs FSA ≈ 0.033). A ternary-reserve advantage is a *prediction* contingent on the untested Gate 0, whose outcome is genuinely open (the format assay measures from-scratch QAT, not a frozen-base reserve).

5 Experiments

Synthetic knowledge-capacity assay: disjoint random fact sets $A \rightarrow B$ (key→label, $V=1024$, so each fact is 10 bits of arbitrary lookup), $K = N \log_2 V - \sum \text{CE} / \ln 2$. A 917k-parameter MLP ($p=0.5$) is loaded with 128k base facts (base recall only 0.744—“saturated” is relative), then adapted. **The reserve is a dense fp32 mask:** this is why everything here is a mechanism demonstration. Single seed; LoRA uses a fixed scale 2.0 (not the standard α/r of Hu et al. [2022]) with its LR tuned down per load to compensate; it also uses AdamW weight decay 0.01 (vs FSA’s 0) and sees a dense base (recall ≈ 0.64) while FSA sees a 50%-masked base (recall 0.744)—so the comparison is indicative, not controlled.

Retention and reserve (Fig. 1). Masked retention is exactly 1.000; full fine-tuning goes below random. The reserve fills to 490.6 kbits and then declines.

LoRA (Fig. 2). With its LR tuned, LoRA learns facts well and at $r \geq 256$ acquires *more* than FSA’s maximum; counted honestly in fp32 it is also *more* storage-efficient. The distinction

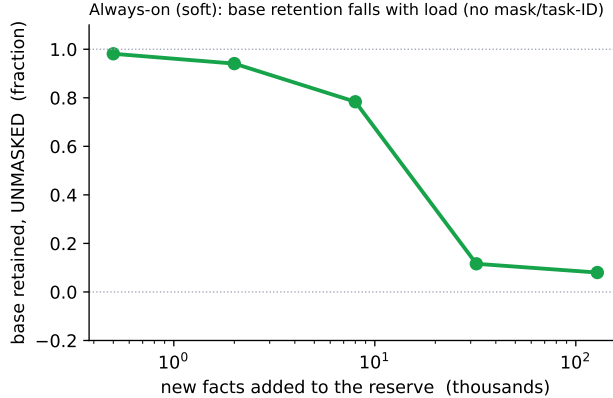


Figure 3: The mask-free soft variant degrades with load (with full base-input replay). It has no addressable slice to delete.

FSA might eventually offer—fewer bits via trits—is a Gate-0 question, still open. We retract the earlier “ $\sim 10\times$ ” claim, which divided fp32 knowledge by hypothetical trit storage.

Soft variant (Fig. 3). Run unmasked, retention *degrades* with load ($0.98 \rightarrow 0.78$ at 8k facts $\rightarrow 0.12$ at 32k), bought with full base-input replay. We describe this as degradation, not “graceful” suitability. The λ weight is a plain retention/acquisition dial.

6 A proposed direction (untested): a revocable patch, and a personal model

What survives review is narrow: if the ternary mechanism works (Gate 0), a quantized model could carry a **revocable domain patch**—task ID in, slice on; drop the slice and the *masked* base returns. A precise statement avoids the overclaims of our earlier draft:

Patch-revocation contract. Deleting an isolated patch exactly removes *that patch’s parameter contribution*. Equivalence to a model trained without its data additionally requires that no retained patch or training state depended on it. It does *not* delete individual facts (facts within a slice are superimposed), and it does not make information already in the base inaccessible.

This is weaker than, and should be compared against, deleting a row from a retrieval store [Lewis et al., 2020, Khandelwal et al., 2020]—which is per-record and leaves the base bit-identical—or disabling a per-user adapter [Tan et al., 2024], which removes that user’s whole contribution without a weight-unlearning problem. The honest case for putting anything in the *weights* is for skills, style, or policy that benefit from parametric adaptation and must act in every forward pass—not for arbitrary facts, which are retrieval’s job and which our own tuned-LoRA numbers memorize more cheaply. A “living personal model” that learns continually and on-device is an appealing target for these properties, but two things remain unestablished and we do not claim them: that personal *skills, style, or policy* (not arbitrary facts, which our assay shows tuned LoRA memorizes more cheaply) benefit from parametric adaptation over retrieval, and that *training* (not inference) fits on-device—the reserve fill is a half-tensor, so AdamW state alone is order 10 GB at a few billion weights. The right evaluation pairs FSA against OPPU-style per-user adapters and local retrieval on a real personalization benchmark [Salemi et al., 2024], with unlearning measured [Bourtole et al., 2021, Maini et al., 2024].

7 Limitations and the deciding experiment

Every number here is a **dense fp32 mechanism demonstration**; the format and real models are untested. **Gate 0:** rerun the assay through the real quantizer—*ternary patches on a pair8 base*, pair occupancy and scales frozen, reserve trained as trits with straight-through estimation (note this is sidecar-style, since an in-codebook block at two pairs cannot open a third). Publish K vs fact count beside the fp32 curve and sidecar bytes beside FP16 and ternary LoRA at matched budget. The outcome is *genuinely open*: the **pair8** format assay measures *from-scratch* QAT at matched loads (its saturation sweep unfinished), so its κ cannot predict a *frozen-base* reserve—and even a pessimistic ternary yield, priced at the corrected ≈ 1.58 bits/trit, need not fall below fp32 LoRA’s 0.057. Gate 0, not the assay, decides. Further: K is a log-loss improvement, not semantic knowledge; κ is assay- and load-specific (at matched load **pair8**’s lower bit-rate gives a *higher* κ than plain ternary; ternary only reaches more *total* knowledge at a higher budget); the frozen base can augment or suppress through new paths but cannot refine.

8 Conclusion

Freezing a disjoint mask preserves the masked function; a quantization reserve memorizes random facts up to a cliff *in the dense fp32 mechanism*; a KL penalty buys a degrading always-on curve. Stated carefully—and only for that fp32 mask—the mechanism is small and true; the ternary reserve that would make it matter is untested. Whether it becomes a *useful* revocable patch—and the substrate for a private model that learns with one person and can *revoke a patch* on request—depends first on the ternary reserve carrying real knowledge at a competitive byte cost, which Gate 0 would decide. That experiment, not another retention-1.000 plot, is the next step.

References

- Zeyuan Allen-Zhu and Yanzhi Li. Physics of language models: Part 3.3, knowledge capacity scaling laws. *arXiv:2404.05405*, 2024.
- Alan Ansell, Edoardo Maria Ponti, Anna Korhonen, and Ivan Vulić. Composable sparse fine-tuning for cross-lingual transfer. In *ACL*, 2022.
- Dan Biderman, Jacob Portes, et al. Lora learns less and forgets less. *TMLR*, 2024. arXiv:2405.09673.
- Lucas Bourtole et al. Machine unlearning. *IEEE S&P*, 2021. arXiv:1912.03817.
- Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. *NeurIPS*, 2023. arXiv:2305.14314.
- Demi Guo, Alexander M. Rush, and Yoon Kim. Parameter-efficient transfer learning with diff pruning. *ACL*, 2021. arXiv:2012.07463.
- Thomas Hartvigsen et al. Aging with grace: Lifelong model editing with discrete key-value adaptors. *NeurIPS*, 2023. arXiv:2211.11031.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. In *ICLR*, 2022. arXiv:2106.09685.
- Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. Generalization through memorization: Nearest neighbor language models. *ICLR*, 2020. arXiv:1911.00172.
- Banseoek Lee et al. Littlebit: Ultra low-bit quantization via latent factorization. *NeurIPS*, 2025. arXiv:2506.13771.

- Patrick Lewis et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *NeurIPS*, 2020. arXiv:2005.11401.
- Zhizhong Li and Derek Hoiem. Learning without forgetting. *TPAMI*, 2017. arXiv:1606.09282.
- Shuming Ma, Hongyu Wang, et al. The era of 1-bit llms: All large language models are in 1.58 bits. *arXiv:2402.17764*, 2024.
- Shuming Ma et al. Bitnet b1.58 2b4t technical report. *arXiv:2504.12285*, 2025.
- Pratyush Maini et al. Tofu: A task of fictitious unlearning for llms. *COLM*, 2024. arXiv:2401.06121.
- Arun Mallya and Svetlana Lazebnik. Packnet: Adding multiple tasks to a single network by iterative pruning. In *CVPR*, 2018. arXiv:1711.05769.
- Arun Mallya, Dillon Davis, and Svetlana Lazebnik. Piggyback: Adapting a single network to multiple tasks by learning to mask weights. In *ECCV*, 2018. arXiv:1801.06519.
- Kevin Meng, Arnab Sen Sharma, Alex Andonian, Yonatan Belinkov, and David Bau. Mass-editing memory in a transformer. *ICLR*, 2023. arXiv:2210.07229.
- Marcin Pietroni et al. Ada-qpacknet: Adaptive pruning with bit-width reduction for efficient continual learning. *ECAI*, 2023. arXiv:2308.07939.
- Andrei A. Rusu et al. Progressive neural networks. *arXiv:1606.04671*, 2016.
- Alireza Salemi et al. Lamp: When large language models meet personalization. *ACL*, 2024. arXiv:2304.11406.
- Joan Serra, Didac Suris, Marius Miron, and Alexandros Karatzoglou. Overcoming catastrophic forgetting with hard attention to the task. In *ICML*, 2018. arXiv:1801.01423.
- Zhaoxuan Tan et al. Democratizing large language models via personalized parameter-efficient fine-tuning. *EMNLP*, 2024. arXiv:2402.04401.
- Vithursan Thangarasa et al. Spdf: Sparse pre-training and dense fine-tuning for large language models. *UAI*, 2023.
- Peng Wang et al. Wise: Rethinking the knowledge memory for lifelong model editing of large language models. *NeurIPS*, 2024. arXiv:2405.14768.
- Xiao Wang et al. Orthogonal subspace learning for language model continual learning. *EMNLP Findings*, 2023. arXiv:2310.14152.
- Bartosz Wójcik et al. Tinysubnets: An efficient and low-capacity continual learning strategy. *AAAI*, 2025. arXiv:2412.10869.
- Yuhui Xu et al. Qa-lora: Quantization-aware low-rank adaptation of large language models. *ICLR*, 2024. arXiv:2309.14717.
- Hattie Zhou, Janice Lan, Rosanne Liu, and Jason Yosinski. Deconstructing lottery tickets: Zeros, signs, and the supermask. *NeurIPS*, 2019. arXiv:1905.01067.